OwnedC

Ownership-Aware Memory Safety for C

without rewriting your codebase

Status: Research Prototype (Pre-1.0)

License: GNU GPLv3

Target Standard: C99 / C11

Executive Overview

OwnedC is a memory safety framework designed for legacy and modern C codebases. It introduces compile-time linting (heuristic borrow checking) alongside a dynamic runtime metadata registry to enforce ownership tracking, scope-bound RAII, and borrow checking. This dual-layer system enables C programs to obtain safety guarantees similar to modern systems languages while retaining interoperability with standard libraries and existing development environments. The system's power is demonstrated directly by plugging it into SQLite (v3.47.0), verifying dynamic leak and double-free detection on an industry-grade codebase.

1. Architecture & Design Goals

Design Principles

- Enable ownership and memory safety without requiring a full rewrite to Rust/Go.
- Provide a zero-overhead allocation path via safe_region (arenas) for performance-sensitive loops.
- Allow hardware-enforced protection (such as CHERI capability pointers) to be adopted incrementally.
- Maintain interoperability with existing standard allocators (jemalloc, mimalloc) and C++ codebases.

Dual-Layer Safety Model

1. Heuristic Static Analysis (ownedc_lint.py):
Performs best-effort, compile-time validation to detect common issues like leaks, double-frees, and use-after-free before the build executes.

2. Dynamic Metadata Registry:
Tracks pointer ownership states (OWNED, BORROWED, SHARED, RELEASED) at runtime. Any violation causes an immediate abort with comprehensive diagnostic dumps.

2. Key Subsystems & Features

* Scope-Bound RAII

Leverages GCC/Clang '__attribute__((cleanup))' to bind the lifetime of heap allocations to local stack scopes. When the scope exits, the compiler automatically triggers deallocation.

* Dynamic Borrow Checker

Enforces single-owner rules at runtime. Tracks active borrows and halts program execution if a pointer is accessed after it is released or modified by another thread.

* Lock-Free Bump Arenas (safe_region)

High-performance memory pools designed for performance-critical hot loops. Bypasses the global lock overhead, providing O(1) bulk deallocations.

* MPSC Channels & Concurrency

Thread-safe Multi-Producer Single-Consumer queues that transfer pointer ownership cleanly across threads, verifying that sending threads can no longer access transferred pointers.

* C++ RAII Wrappers

Header-only wrappers ('ownedc.hpp') defining templates like 'owned_ptr<T>' and 'borrowed_ptr<T>' for clean integration into modern C++ projects.

* Bare-Metal / Kernel Mode (NO_STDLIB)

A lightweight build configuration that strips dependency on standard POSIX APIs, disabling multi-threading, sockets, and deallocating in static block pools.

3. Performance Characteristics

Dynamic tracking incurs overhead due to registry lock contention. Below are benchmark results for 500,000 allocations of 32 bytes each on Apple Silicon (macOS, Clang 15, -O3):

Strategy	Single-Thread	S-T Overhead	8-Thread	8-T Overhead
Raw malloc / free	0.0616s	1.0x (Baseline)	0.0296s	1.0x (Baseline)
owner_malloc / free (Tracked)	9.2519s	150.1x	5.5906s	188.7x
safe_region (Arena)	0.0097s	0.15x (6.3x faster)	N/A	--

Comparison to Alternative Safety Models

- Checked C (Microsoft): Requires custom pointer syntax and heavy code modifications.
- ASan (AddressSanitizer): Highly useful for debug builds, but too slow for production.
- Rust: Offers zero-cost compile-time safety, but requires a complete rewrite of legacy code.
- OwnedC: Standard C syntax, modular adoption, and optimal performance via arenas.

4. Real-World Verification: SQLite Showcase

To verify OwnedC under complex, production-grade workloads, we integrated it directly into SQLite (v3.47.0). By implementing a custom 'sqlite3_mem_methods' structure, we redirected all internal SQLite database heap allocations (including page caches, parse trees, and b-trees) to OwnedC's dynamic registry (owner_malloc, owner_free, owner_realloc, and owner_malloc_usable_size).

We ran a full database operation (creating a table, inserting records, and executing prepared queries) and analyzed three execution scenarios:

* Normal Database Run

During normal execution, SQLite makes 196 heap allocations. Upon closing the connection and statement handles, OwnedC reports exactly 0 leaks and 196 freed allocations, proving that SQLite runs perfectly and cleanly under the OwnedC memory safety layer.

* Leak Detection Showcase

Running with '--simulate-leak' (forgetting to call sqlite3_finalize on stmt handles before exit) causes the registry to immediately flag and dump all 58 unreleased allocations, showing the precise address and size of each leaked database resource.

* Double-Free & Use-After-Free Prevention

Running with '--simulate-double-free' (attempting to release an SQLite pointer twice) triggers an immediate runtime intercept. OwnedC prints a detailed traceback ('Reason: Double-free') and safely aborts execution, successfully blocking memory corruption exploits at the database memory boundaries.

Roadmap & Future Direction

1. Fix hash-map collision behaviors to reduce owner_malloc global lock contention.
2. Add thread-safe Multi-Threaded Arenas.
3. Complete external security audits ahead of the official 1.0 release.